

Cole Scheller

[Email](#) · (661) 619-4654 · [Portfolio](#) · [GitHub](#) · [LinkedIn](#)

Education

Michigan State University Sept. 2022 - May 2026
Bachelor of Science, Computer Engineering / Minor in Mathematics
Dean's list 2022-2025; '24-'25 College of Engineering Undergraduate Service Award
GPA: 3.5

Experience

CANVAS — Michigan State University May 2023 - May 2026
Undergraduate Research Assistant

- Led group dedicated to labelling a subset of collected dataset samples to demonstrate the necessity of a multi-season & multi-modal dataset, contributing to the acceptance of the [MSU-4S](#) dataset at CVPR
- Modified existing open-source pointcloud annotation tool LabelCloud to better suit labelling team's needs by altering the pointcloud rotation algorithm to be more intuitive, adding visual aids to guide the labeller's understanding of the 3D scene, and implementing additional keyboard shortcuts and bounding-box input methods, decreasing the labelling time per scene by approximately half
- Designed and executed experiment to evaluate efficacy of repurposing existing dedicated short-range communication (DSRC) equipment for dedicated over-the-air sensor data sharing, assisting further development of cooperative perception for rail application

MCity — University of Michigan May 2024 - Aug. 2024 & May 2025 - Aug. 2025
Engineering Intern

- Improved responsiveness of garage door state updates within internal application through the use of an MQTT broker and service, requiring containers through AWS's ECS and changes to internal API's legacy codebase, bringing door state update times from at most thirty seconds to at most two seconds
- Worked to decrease the time taken to boot roaming real-time kinematic (RTK) quasi-embedded units using armbian, requiring both the usage of "systemd" to analyze most consequential system startup jobs, as well as alterations to system's boot environment, resulting in a boot time reduction of almost 25%
- Replaced RTK units' manual update process with an automated over-the-air procedure by converting the units' functions into containerized docker services which are automatically built and uploaded to AWS's elastic container registry (ECR) using AWS CodeBuild and CodePipeline, then configuring the units to retrieve container images from ECR upon startup
- Designed and implemented testing infrastructure for legacy project involving flask and SQLAlchemy, using pytest, allowing incremental test additions to gradually grow test coverage from zero
- Integrated automatic reST-based documentation generation via Sphinx into legacy project, leveraging AWS CodePipeline and CodeBuild to automatically build and include documentation upon repository updates without impacting existing source control workflows

Autonomous Vehicles Club — Michigan State University Aug. 2022 - Apr. 2026
Transition Adviser (2026), President (2023-2025), Perception Team Lead (2023-2026)

- Transitioned leadership to incoming executive team while mentoring successors, improving year-end retention
- Worked with leadership team to bring club back to pre-COVID standards, organizing the efforts to handle developing new student onboarding material, social media advertisement, and administration tasks required by the university
- Developed RGB Camera-to-LiDAR pointcloud projection in C++ using LibPCL and integrated with Robot Operating System (ROS) using TF2 to transform between LiDAR, camera, and world coordinate spaces

ApprenTek — Dexter, MI Jan. 2021 - Sept. 2021
IT Intern

FIRST Robotics Competition — Dexter, MI Sept. 2018 - Apr. 2022
Team Captain (2020-2022), Computer Vision Team Lead (2018-2022)

Selected Projects

IndyTUI
A terminal-based telemetry display, written in C++, designed to follow INDYCAR races live, live with a delay, or on a later date via recording & replay.

- Designed & implemented thread-safe circular telemetry queue with a calculated size given telemetry refresh rate and desired delay, allowing for a delayable queue with a memory footprint fixed given the delay and refresh rate
- Leveraged the delay queue's circular design & thread safety to permit new telemetry received to update the head of the queue, only advancing the head whenever the delay was no longer satisfied, ensuring a satisfied delay is prioritized over continuity of received telemetry
- Deconstructed telemetry API by monitoring a local live session in existing tooling with [mitmproxy](#), observing the endpoints and payloads for session initiation

AgentLang

A tree-walk interpreter for a simple scripting language designed to instruct the actions of simulation agents in varying simulation contexts. Designed for capstone project.

- Implemented two-stage parsing process, performing most standard parsing steps in the first stage while deferring some, like the linking of a function call to the relevant body AST node, until finalization, permitting more complex yet necessary language behavior like function recursion
- Designed and implemented the underlying simulation's agent action API using C++'s variant, allowing for a straightforward visitor based approach to action execution, and decoupling agents' action expression from simulation context and agent type

LabelCloud

A forked PyQt based tool to assist in the labeling of pointclouds. Modifications allow both labeling pointclouds, as well as calibrating camera extrinsics.

- Significantly narrowed scope of strategies within existing strategy-based design, allowing a greater number of UI/UX elements to function agnostic of usage-mode
- Rearranged interface, adding QT DynamicProperties to each element to control event calls and usage-mode-dependent behavior, increasing maintainability and readability by removing a necessity for independent element initialization logic
- Added image display to main interface with zooming, panning and pixel selection, allowing for manual correlation of pointcloud points to image points

Skills

Languages: C++, Python, Bash, GNU Emacs Lisp, LaTeX

Frameworks/Libraries: NumPy, OpenCV, Flask, ROS1, ROS2, TF2, MQTT

Cloud & DevOps: AWS ECS, CodePipeline, CodeBuild, Docker, Git, Linux